

Reviewer Pack — Schur-Weyl Multiplicity Gap in Permanent vs Determinant Tens...

Entry 27563135d273 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-01 12:03:53 UTC

Schur-Weyl Multiplicity Gap in Permanent vs Determinant Tensor Powers

Entry ID: 27563135d273 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-01 12:03:53 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): 3-SAT Instances

Statement:

For any 3-SAT instance with n variables, the multiplicity of the irreducible representation $\lambda = (n-1, 1)$ in the decomposition of the permanent polynomial's tensor power $P^{\otimes k}$ is strictly greater than that in the determinant polynomial's tensor power $D^{\otimes k}$ for all $k \geq 2$, where P and D are homogeneous of degree n .

Rationale (proposer's reasoning):

Schur-Weyl duality links symmetric group representations to polynomial invariants. The permanent's symmetric nature under row/column permutations creates higher multiplicities for certain Young diagrams compared to the determinant's antisymmetric structure, potentially exposing algebraic barriers to efficient computation.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bbcd0a07277eff2f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_3sat_clause_valid(clause):
        return len(clause) == 3 and all(isinstance(lit, int) and lit != 0 for lit in clause)

    def generate_random_3sat_instance(n: int) -> list:
        clauses = []
        for _ in range(2 * n): # Each variable appears twice
            while True:
                clause = [random.choice([-1, 1]) * random.randint(1, n)]
                if is_3sat_clause_valid(clause):
                    clauses.append(clause)
                    break
            return clauses

    def hook_length_formula(n: int) -> float:
        total = 0
        for i in range(1, n + 1):
            for j in range(1, n - i + 2):
                total += (n - i + 1 - j) * (n - j + 1)
        return math.factorial(n) / total

    def plethysm_coefficient(n: int, k: int) -> float:
        coeff = 0
        for m in range(1, n + 1):
            coeff += ((-1) ** (n - m)) * hook_length_formula(m) ** k
        return coeff / hook_length_formula(n)

    def multiplicity(poly_tensor_power: int, lambda_):
        return plethysm_coefficient(lambda_, poly_tensor_power)

    n = random.randint(5, 20)
    P_k = multiplicity(n - 1, 2)
    D_k = multiplicity(n - 1, 2)
```

```

if P_k > D_k:
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "Permanent's multiplicity is not strictly greater than Determinant's"

return {
    "metric_name": "Multiplicity Gap",
    "metric_value": P_k - D_k,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6009679.py", line 75, in <module>

result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6009679.py", line 51, in run_trial

P_k = multiplicity(n - 1, 2)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6009679.py", line 48, in multiplicity

return plethysm_coefficient(lambda_, poly_tensor_power)

^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6009679.py", line 44, in plethysm_coef

```
coeff += ((-1) ** (n - m)) * hook_length_formula(m) ** k
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d6009679.py", line 39, in hook_length_f
    return math.factorial(n) / total
           ^^^^^^^^^^^^^^^^^^^^^
```

ZeroDivisionError: division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing evaluation of the conjecture's validity. |
next: Fix hook_length_formula implementation to handle edge cases in partition calculations

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109364
2	propose	ollama_remote	qwen3:8b	0	0	54718
3	preregistration	ollama_remote	qwen3:8b	0	0	24247
4	novelty	ollama_remote	qwen3:8b	0	0	20799
5	novelty	ollama_remote	qwen3:8b	0	0	15439
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10989
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9828
8	judge	ollama_remote	qwen3:8b	0	0	20739

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 266123 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in *research/audit/27563135d273.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/27563135d273.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/27563135d273.tar.gz* (if generated)